brief-fullstack

Case Study: Fullstack Product Engineer

Welcome, and thank you for taking the time to join our selection process. This document and the platform repository contain everything you need to complete the assignment. There is no separate back-and-forth required. Read this document once fully before starting. If something is ambiguous, treat that as part of the exercise: make a reasonable assumption, write it down in your report, and proceed.

---

1. Role Expectation: We Are Hiring a Product Engineer

This is the section to read twice, because it decides everything else in this brief.

A conventional software engineer waits for a specification, implements it, and is judged on whether the code works. We are not hiring for that. We are hiring someone accountable for whether the feature is worth shipping at all, and who will learn whatever is needed to ship it well: an unfamiliar framework, an unfamiliar domain, a regulation, a business model, and the real working conditions of the people who use it.

We expect you to think from first principles:

- Take the problem down to what is genuinely true.
- Ask what this product should be if nobody had built it yet.
- Decide what to change given what exists in the codebase.
- Decompose and rank your reasoning so reviewers can follow it and evaluate your choices.

The change we are most interested in is the one that makes this product genuinely better for the people it touches, not the one that touches the most lines of code.

Monozukuri: Craftsmanship Beyond the Minimum Baseline

Every requirement, guideline, and evaluation point stated in this brief represents the **minimum baseline**. We do not view this exercise as a mere take-home test, but as your canvas to show off your highest standards of engineering and product craftsmanship (**Monozukuri**).

We expect you to bring:

- **Pride in Making (Monozukuri)**: Deep care for the product, meticulous attention to detail, and a drive to craft software that feels complete and reliable.
- **Great UI/UX Design Taste**: Clean layout hierarchy, modern visual aesthetics, intuitive micro-interactions, responsive states, and thoughtful design details that make using the application a delight.
- **Uncompromised System Rigor**: Clear architectural thinking, robust planning, and structured reasoning from data models to client-side presentation.

Treat the baseline as your starting floor, not your ceiling. Show us what "top-notch" looks like in your hands.

---

2. Step-by-Step Execution Guide

To make your journey smooth and structured, follow these steps sequentially from setup to submission:

```
Step 1: Setup & Local Exploration
   ↓
Step 2: Deep Context & Domain Immersion
   ↓
Step 3: Defining Problem & Gap to Ideal Condition
   ↓
Step 4: Revamp Strategy, Acceptance Criteria & Trade-offs
   ↓
Step 5: Monozukuri Implementation & Pull Request
   ↓
Step 6: Final Submission (Single PDF & Artifacts)
```

Step 1: Setup & Local Exploration

Repository: `github.com/rakamindev/ai-interview-platform`

The repository holds both services: `api/` (Rails on PostgreSQL with Sidekiq, RSpec, and Gemini AI) and `web/` (React 18, TypeScript, Vite, Tailwind, and Radix UI). Each directory contains a `README.md` for running it locally.

1. **Clone & Run**: Clone the repository and run both services locally. Explore the current workflow in your browser and test it in your own way.
2. **Branch**: Create a feature branch off `main`. Do not commit directly to `main`. Keep commits small and readable.
3. **Understand the Baseline**: RSpec currently has no specs written, the web app has no test runner, and there is no continuous integration. Transferring fast into an unfamiliar stack and building enough testing harness to prove your work is part of owning the change.

Step 2: Deep Context & Domain Immersion

Before proposing changes or writing code, build context around these 5 core pillars:

1. **The Product**: Use it end to end before you judge it. Read what it does, not what the code implies it does.
2. **The Industry**: Hiring and talent assessment in Indonesia. Understand what makes that hard, what is already commoditized, and where the real leverage sits.
3. **What It Is For**: The outcome this product exists to produce, what has to be true for it to keep being useful, and what would make it matter to more people than it reaches today.
4. **The Users**: Assessors, recruiters, and hiring managers. Understand what their day looks like and what they are actually trying to get done.
5. **The People Affected Who Never Chose It**: Candidates are assessed by this product. They do not pick it, they cannot opt out of it, and a wrong result changes a real person's year. Design for them too, keeping Indonesia's Personal Data Protection Law (UU PDP) in mind.

Step 3: Defining Problem & Gap to Ideal Condition

Walk the real workflows end to end, inspect the database records, and evaluate the API payloads to identify the **gap between the current flawed implementation and an ideal, top-notch product condition**.

Done when:

- You clearly articulate the core problem and the gap preventing this product from delivering tangible value to users (assessors, recruiters, hiring managers) and candidates.
- Findings are categorized by severity (P0 to P3) with a one-line impact statement showing how the current flaw harms the real user workflow or candidate evaluation.
- You separate **missing specification** (never defined) from **defective implementation** (defined but broken).
- Findings span both services (`api/` and `web/`), including the data computation and frontend presentation seam.
- **Constraint Signal**: You identify and document early any blocking risk, ambiguity, or architectural debt that you would escalate to a Technical Lead on a live project.

Step 4: Revamp Strategy, Acceptance Criteria & Trade-offs

Design the revamp or enhancement that directly bridges the gap to the ideal product condition, covering **both a backend module and a frontend module**. Depth on one side is expected, but complete absence on the other is not.

1. **Derive Self-Defined Acceptance Criteria**: Write down what correct behavior means for every input and edge case before writing code (handling unassessed skills, missing ratings, model call failures, edge-case data types, and long text states).
2. **Evaluate Solution Options & Trade-offs**: Compare at least two or three technical options (Option A vs Option B) to ship this revamp:
   - **Product Impact vs Cost**: What each option buys to transform the user experience into a top-notch product, what it costs, and what it forecloses.
   - **Long-term Maintainability**: Who maintains it, how easily it can be evolved, and whether it can be walked back cheaply.
   - **Failure Modes**: What conditions break it under stress or bad inputs.
   - **Contextual Fit**: Why your chosen option is the best way to deliver tangible, high-impact value under **this** codebase and **this** deadline.

Step 5: Monozukuri Implementation & Pull Request

Land your changes across the full stack: the API, the data model underneath, the frontend screen, and automated tests.

Minimum Baseline Standards

- **Proven Correctness**: A test fails if the behavior regresses, and you have watched it fail.
- **Designed Failure Paths**: Handle timeouts, partial writes, model errors, and duplicate jobs explicitly.
- **Protected Data**: Migrations must be reversible and safe against existing rows. Endpoints authorized, inputs validated, and no sensitive personal data in logs or commits (UU PDP compliance).
- **Polished UI/UX Interface**: Great visual taste, harmonious layout, handling all interaction states (loading, empty, error, partial, long text, and responsive views) using or extending the design system.
- **Seeded Fault Test**: Prove your tests are real by breaking your logic on a scratch branch to show the test catch it, then reverting with history visible.
- **AI Verification Moment**: Document at least one instance where AI code generation was wrong or risky and how you verified/corrected it.

Pull Request Options

You can structure your GitHub submission as either:

- **Option A**: A single, comprehensive Pull Request containing the full change across both services.
- **Option B**: An **Umbrella PR** setting out the overall vision and linked to a few focused sub-PRs.

Step 6: Final Submission

You have until **Wednesday 19 August, 13:00 WIB** to submit your work.

What to Submit

Upload a **single PDF document** in the hiring platform portal.

- **Flexible Format & Style**: The design, template, and formatting of your report are completely flexible and open to your personal documentation style.
- **Step-by-Step Narrative Required**: Your report must clearly present your execution journey step by step, reflecting how you navigated Steps 1 through 5 to deliver a meaningful product transformation.

Report Checklist (Minimum Baseline Standards)

Your PDF report must mirror your step-by-step journey across the brief:

1. **GitHub Pull Request Link**: Link to your open Pull Request(s) on `github.com/rakamindev/ai-interview-platform` (Option A single PR or Option B Umbrella PR).
2. **Written Documentation & Execution Narrative**:
   - **From Step 2**: Product context & domain immersion insights (users, candidates, UU PDP implications).
   - **From Step 3**: Problem & gap analysis to ideal condition (severity-ranked P0 to P3 findings, workflow impact statements, and your constraint signal).
   - **From Step 4**: Revamp strategy & trade-off evaluation (Option A vs Option B rationale, product impact vs cost, maintainability) along with your self-derived acceptance criteria and edge cases handled.
   - **From Step 5**: Monozukuri execution proof (test coverage evidence, seeded fault test proof, AI verification moment, and claimed engineering depth).
3. **Visual Screenshots**: Embedded images showcasing the revamped UI flows, edge cases, error states, and responsive views.
4. **Video Demonstration Link**: A link to a 3 to 5 minute video walkthrough hosted on an external platform (Loom, YouTube Unlisted, Google Drive, or similar) demonstrating the end-to-end user flow, problem clarity, and feature enhancements.
5. Add more if you like.

---

3. Dual Evaluation Structure

Your submission is evaluated by two distinct teams:

1. **Product Team**: Reviews what you ship in your report (problem clarity, user flow empathy, documentation, screenshots, UI/UX design taste, and video demonstration).
2. **Engineering Team**: Reviews what you ship in code (GitHub Pull Request(s), code architecture, test suite, data migration safety, and trade-off reasoning).

4. Evaluation Rubric

- **Product Impact & Monozukuri Craft (50%)**: What you actually ship to transform this product into a meaningful, top-notch experience. Evaluated by the impact of the problem solved, trade-off rigor, self-derived acceptance criteria, exceptional UI/UX design taste, and tangible value brought to users and candidates.
- **Engineering Craft & Execution (50%)**: The correctness of the shipped fullstack slice, test harness reliability, edge case handling, data safety, clean code architecture, and PR quality.

---

5. Non-Negotiable Disqualifiers

Any of the following results in an immediate decline:

- Weakening or removing a test assertion to make a check pass.
- Committing secrets, API keys, or real personal data.
- Submitting code you cannot explain during the technical interview.
- Submitting a change that was only tested on the happy path.
- Reproducing another candidate's visible work without attribution.
- Poor or neglected UI/UX design quality.

---

6. Ground Rules & Live Defense

- **Inputs**: You receive this brief and the repository. Scope, criteria, edge cases, and test data are derived by you.
- **AI Tooling**: AI is encouraged. Use it as leverage you verify, not an oracle you trust blindly.
- **No Live Q&A**: Make reasonable assumptions, document them in your PDF report, and proceed.
- **Live Technical Defense**: A 45 to 60 minute video session with the CTO and Technical Lead after the deadline to discuss your trade-offs, architectural choices, and rejected options.

The platform is realistic and imperfect. Show us what you can ship to make it meaningful. Good luck!